Spec MQTT-next

William Yang

March 5, 2024

Contents

1 Abstract **3**

2 Definitions **3**

3 Notational Conventions **4**

4 Motivation **4**
4.1 Performance Enhancements . . . 4
4.2 Network address migration . . . 4
4.3 Reliable Delivery in Unreliable Networks . . . 4
4.4 Multiplexing enables flexible asynchronous parallel streaming . . . 5
4.5 Flexiable message delivery . . . 5
4.6 Embedded security . . . 5
4.7 Pluggable security suite and congestion control . . . 6

5 New features in MQTT-next **6**

6 Overview of changes/extensions to the MQTT protocol **9**

7 Operating Modes **10**
7.1 Single Stream . . . 10
7.2 Simple multistreams . . . 11
7.3 Advanced multistreams . . . 11
7.4 Work mode feature summary . . . 13

8 Connections **14**
8.1 Establishing a connection . . . 14
8.2 Connection Keepalive . . . 14
8.3 Connection termination . . . 14
8.3.1 Graceful shutdown . . . 14
8.3.2 Abnormal connection shutdown . . . 16
8.3.3 Sending unreliable datagrams over the connection . . . 16
8.3.4 Connection downgrade . . . 18
8.3.5 Discovering and upgrading . . . 18

9 MQTT Flows **19**
9.1 Flow usages . . . 20
9.1.1 Control Flow . . . 20
9.1.2 Data Flow . . . 20
9.2 Flow and stream mapping . . . 21
9.3 Flow ownership . . . 21
9.4 Flow ID . . . 21
9.5 Flow Type . . . 21
9.6 Flow Header . . . 22
9.6.1 Control Flow Stream header . . . 22
9.6.2 Client Data Flow Stream header . . . 22
9.6.3 Server Data Flow Stream header . . . 22
9.6.4 User defined Flow Stream header . . . 22
9.7 Flow Expire Interval . . . 23
9.8 Flow Flags . . . 23
9.9 Optional Headers . . . 23
9.10 Flow start . . . 24
9.11 Send/Recv over the flow . . . 24
9.12 Flow expiration . . . 25
9.13 Flow Termination (Shutdown) . . . 25
9.13.1 Flow graceful termination. . . . 25
9.13.2 Flow abortive termination. . . . 26
9.14 Flow takeover . . . 26
9.15 Flow Recover . . . 27
9.16 Discard the Flow state at peer . . . 28
9.17 Flow state machine . . . 28
9.18 Seq chart of graceful shutdown . . . 30
9.19 MQTT Packet and Flow mappings . . . 30
9.20 Table of Flow Types . . . 31
9.21 Flow State . . . 31
9.21.1 Client side . . . 32
9.21.2 Server side . . . 32

10 Session **32**
10.1 Session State . . . 32

11 Error Handlings **33**

12 Error Code **34**
12.1 Application Error Code on Connection . . . 34
12.2 Application Error Code on Stream Flow . . . 34
12.3 Error Code in MQTT Packet . . . 34

13 Limitations **35**

14 IANA Considerations **35**

15 Opportunities **35**
15.1 Enable MQTT Stream publish mode. . . . 35

1 Abstract

This document defines ‘MQTT-next‘, a mapping of MQTT to the QUIC transport protocol.

2 Definitions

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “NOT RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in BCP 14 RFC2119 RFC8174 when, and only when, they appear in all capitals, as shown here.

`endpoint`: MQTT client or MQTT broker

`connection`: A transport-layer connection between two endpoints using QUIC as the transport protocol, defined in RFC9000

`stream`: QUIC stream defined in RFC9000

`session`: MQTT session defined in MQTT spec

`bidi`: stream direction, bidirectional

`unidi`: stream direction, unidirectional

`local stream`: stream which is initiated locally

`remote stream`: stream which is initiated by remote endpoint.

`flow`: stream abstraction in MQTT layer, part of MQTT session.

`flow owner`: the endpoint who initiates the stream thus owns it.

`client flow`: flow initiated by the client and client owns it.

`server flow`: flow initiated by the server and server owns it.

`subscriber`: client which does subscription with topic-filter.

`publisher`: client which publish application messages to the server.

`datagram`: QUIC unreliable datagram

`sender`: endpoint which can send data over a flow

`receiver`: endpoint which can receive data over a flow

`MQTT packet(s)`: MQTT control packets defined in MQTT protocol.

`HOLB`: head-of-line blocking

`peer`: remote endpoint

`MQTT-next`: unversioned MQTT version that supports MQTT over QUIC advance multi-stream mode.

`ttl`: time to live

3 Notational Conventions

Conforms to 1.3 of the notational conventions of RFC 9000

4 Motivation

QUIC is the transport layer of HTTP/3, standardised by IETF.

MQTT could leveraging QUIC's powerful features promises to significantly enhance MQTT's flexibility and performance,
particularly in the context of modern mobile networks and demanding applications.

4.1 Performance Enhancements

Faster connections: QUIC's single round-trip connection establishment significantly reduces latency, especially for clients on high-latency networks.

Rapid reconnection and early data sending: QUIC's 0-RTT resumption allows for near-instantaneous reconnection after disconnections, while early data sends eliminate waiting for server responses.

4.2 Network address migration

QUIC seamlessly adapts to network changes, ensuring MQTT clients remain connected and minimizing offline periods.

4.3 Reliable Delivery in Unreliable Networks

QUIC is specifically designed to operate effectively in networks with packet loss and reordering.

This is crucial for MQTT-next in mobile network scenarios where such network conditions are common.

4.4 Multiplexing enables flexible asynchronous parallel streaming

Stream runs independently, single stream provides ordering and multistream provides concurrency.

- Dynamic flow control and prioritization

  Application data is managed effectively with precise flow control and prioritization features.

- Coexistence with other public/private protocols such as http/3.

- Packet send/recv cancellation.

  Large payload message in transmission could be cancelled by either receiver or sender.

- Make MQTT more resilient to application errors.

- Mitigate HOLB

  Head-of-Line Blocking (HOLB) only affects specific QUIC streams, not the entire connection, minimizing its overall impact.

4.5 Flexiable message delivery

Delivery Options: QUIC offers a spectrum of delivery options, including ordered/unordered and reliable/unreliable, catering to diverse application requirements.

4.6 Embedded security

- Default TLS 1.3:

  QUIC utilizes the latest TLS 1.3 by default, offering strong encryption, perfect forward secrecy, and improved performance compared to older versions.

- Post-Quantum Cryptography (PQC) Readiness

QUIC is designed to readily integrate PQC algorithms when they become standardized, ensuring long-term cryptographic agility against potential quantum computing threats.

- Key Update Mechanisms:

  QUIC employs robust key update mechanisms, including forward secrecy and session resumption, to mitigate replay attacks and maintain security even after key compromises.

- Integrity and Authentication:

  QUIC employs authenticated encryption, ensuring both data integrity and sender authentication, preventing unauthorized message modifications and impersonation.

- Cryptographic Integrity:

  Even in the presence of packet loss or reordering, QUIC's cryptographic mechanisms ensure message integrity and authenticity.
  This prevents unauthorized data modifications and protects against potential security exploits.

- Denial-of-Service (DoS) Protection:

  QUIC incorporates several features to mitigate DoS attacks, such as connection limits, packet pacing, and handshake throttling.

4.7 Pluggable security suite and congestion control

Always ready for future updates without requiring major changes to the network.

Congestion control can be tailored to the needs of the application.

5 New features in MQTT-next

- Fast security handshake with 1 RTT and 0 RTT

  Secure connection handshake could be done in 0 or 1 roundtrip time.

- Connection could survive network changes.

QUIC's address migration makes MQTT more robust to network changes, reducing the chance of disconnection.

- Elimination of HOL blocking.

  In TCP-based transport, the MQTT packet at the head of the line blocks all subsequent messages following it, it also
  blocks the MQTT.PING/MQTT.PINGREQ for keepalive.
  Long blocking of keepalive could cause disconnection at other endpoint.

  With QUIC, QUIC knows the importance of each message and sends them in separate channels that won't block each other.

- Separate control and data traffic.

  With TCP-based transport, a MQTT.PUBLISH message with a large payload can block the entire TCP stream and MQTT.PINGREQ/MQTT.PINGRESP.
  With QUIC, the PUB message and the PINGREQ could be sent in different streams.

  PINGREQ, which is used for keep-alive or liveness detection at the MQTT layer, must be sent on a higher priority control flow.

- Classified application data

  QUIC multi-streams allows the application to send different application data on different streams.

  For example

  1. assign different topic data to different streams

  2. Separate stream for different QoS messages.

  3. Separate stream for publishing and subscriptions.

- Flow control on classified traffic

  QUIC enables flow control both at the connection level and at the stream level.

  This allows application data relays on different QUIC streams to be flow controlled independently.

- Prioritised traffic

  QUIC enables MQTT to prioritise traffic from different streams.

  This also affects loss recovery behaviour and network congestion.

- Enhanced security

- Coexistence with other applications on the same connection such as HTTP/3

  QUIC Multiplexing allows the MQTT protocol to coexist with other public/private protocols on the same connection.

- MQTT packet(s) transmission could be cancelled.

  QUIC makes it possible to abort a MQTT packet on both the sender and receiver side without affecting the connectivity.

  For cases like

  - Cancel the transmission of a large payload packet.

  - Cancel the transmission of obsolete packets.

  For TCP-based traffic, cancelling a pending MQTT packet means disconnecting and reconnecting.

- Support both reliable and unreliable delivery.

  RFC9221 extended the QUIC protocol to support unreliable delivery.

  This could make MQTT QoS 0 packets truly "fire and forget" with almost no cost for retransmission.

  In TCP-based protocol, the TCP segment containing the bytes of the QoS 0 packet is retransmitted by the TCP stack in order.

- Build-in transport layer keepalive

In MQTT-next, both client and server could use the keep-alive mechanism of QUIC transport, which is end-to-end.

This simplifies the implementation at the MQTT client and server in terms of timing.

And it is end to end, meaning that the keepalive message must be delivered to the peer without worrying about being terminated
through a middleman such as a proxy, NAT gateway or LB.

- Failure isolation.

  The client and the broker can agree how to handle a failure per flow. To minimise the side effect of the failure.

  A single messaging failure such as a malformed packet MUST cause the flow to be aborted, but it MAY or MAY NOT cause the connection to be closed.

- Variable header compression [TBD]

  MQTT packets are binary coded packets, it is designed for smaller packet size. In order to reduce packet size without losing information,
  topic alias could be used to avoid retransmitting whole long topic in each packet. But that is not all for the other headers, such as the Content Type header.

  HTTP/3 Q-PACK enables header compression/encoding, which the MQTT protocol could use to reduce packet size by compressing other variable headers,
  variable headers or user-defined properties.

6 Overview of changes/extensions to the MQTT protocol

1. MQTT packets are transported via reliable flow or unreliable datagrams.

2. The subscription is now associated with the flow.

3. Acking QoS > 0 messages is also done on the same flow that it is published.

4. Publish QoS 0 messages MAY have the Packet ID field as they could be sent in datagrams. Application at receive side MAY use Packet ID to identify if the packet is a resend or check the ordering of
   unordered messages.

5. Flow state per flow is introduced to track the QoS > 0 message delivery.

6. MQTT packet flow control is now in the flow scope instead of in the connection scope. The flow header could have optional "Receive maximum" header.

7. The server can 'push' messages to the server flow, which the server initiates.

8. PINGREQ/PINGRESP are associated with the flow for application liveness detection, and the keep-alive interval is not enforced on the data stream.

7 Operating Modes

A QUIC connection is REQUIRED between the client and the server as defined in RFC 9000.

The MQTT packets are transported over the flows, which are the QUIC streams.

A QUIC stream provides reliable in-order delivery of bytes, but makes no guarantees about the order of delivery of bytes on other streams.

QUIC streams can be either unidirectional, carrying data only from the initiator to receiver, or bidirectional, carrying data in both directions. Streams in the connection can be initiated by either endpoint, the client or the server.

There are three modes of operation for QUIC-next, each mode having its own advantages and disadvantages in terms of

- Compatibility with MQTT protocols

- Supported features

7.1 Single Stream

The simplist mode simply replaces the TCP based transport with a QUIC stream in the QUIC connection.

A BIDI stream is initiated from the client after the connection handshake and is used to carry all MQTT
to carry all MQTT control packets. It is compatible with MQTT 3.1 and MQTT 5.0 and nothing in the MQTT packet is
changed in the MQTT packet.

Pros: Easy to implement, NO changes in MQTT layer. Benefits from QUIC connection.

Cons: For complex applications that have multiple topics and/or different QoS,
Does not take full advantage of QUIC transport features.

7.2 Simple multistreams

Enhanced single stream mode with support for multistreams, i.e. one control stream and one or more data streams.

Application data and QUIC stream mapping is controlled by the client.

Compatible with single stream mode.

Advantages:

1. Support for multiple streams.

2. Mitigate HOLB application side.

3. Enable parallel processing at both endpoints.

4. Sender defines priority.

5. Freedom in application data and stream mapping

Disadvantages:

1. Persistent data stream session is not available on data stream.
   In this mode there is no stream header, the stream only streams MQTT packets, client and server could not recover
   the data stream states from disconnection.

7.3 Advanced multistreams

Extends the simple multistream mode with the following features:

1. Can coexist with another protocol (http/3 or private protocol) on the same connection.

2. Support unreliable delivery.

3. Defines control message cancellation procedure.

4. Optionally use server initiated stream for predefined subscriptions.

5. Abstract 'flow' concept that could be resumed after reconnect.

6. Q-PACK support for message header compression, greatly reducing message size. @TODO

7. Defines robustness flow procedure.

8. Defines protocol discovery and upgrade/downgrade procedure.

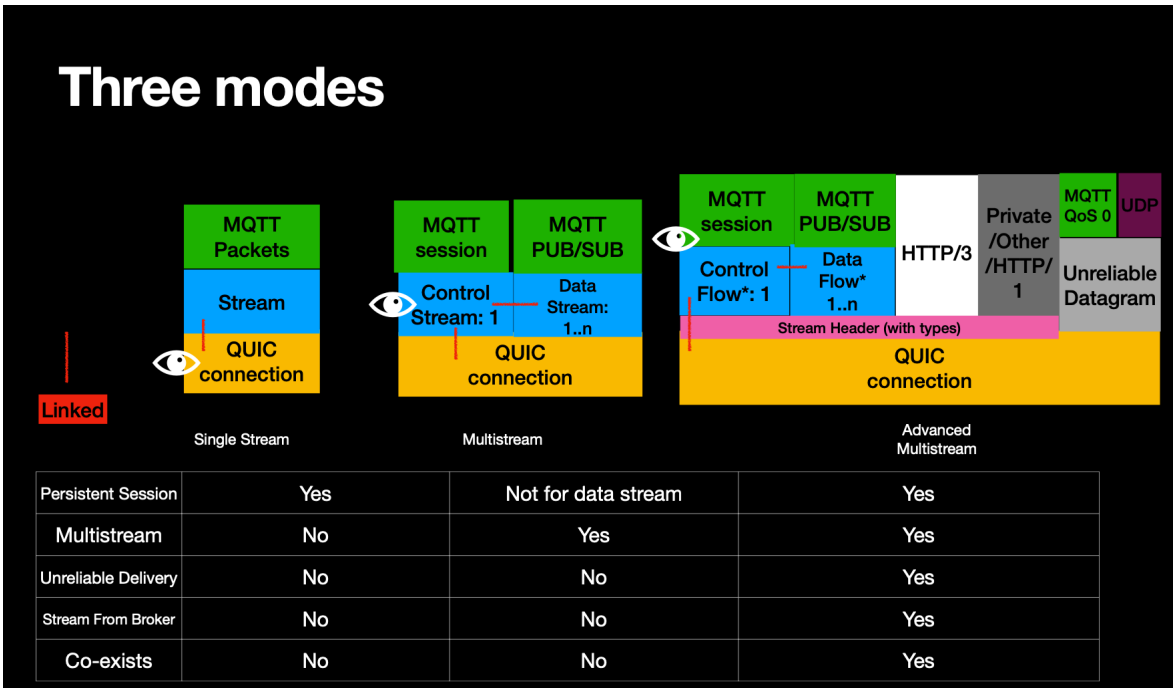
Advantages:

- MQTT 5.0 feature complete
- Flexible packet delivery reliable/unreliable, ordered, out-of-order, send/recv abortions.
- Flexible control stream discovery.
- Flexible connection management.

Disadvantages:

- Extends MQTT 5.0 session data, requires changes to MQTT session layer.
- Fallback to TCP/TLS becomes a completely different protocol.

7.4 Work mode feature summary



Mode	Single Stream	Simple Multstreams	Advanced Multstreams	notes
MQTT 3.1	Y	Y	N	
MQTT 5.0	Y	Y (Partly)	N	
MQTT-next	N	N	Y	
TLS alpn	mqtt	mqtt	MQTT-next	
Connection features				
Transport Keepalive	Y	Y	Y	
1 RTT / 0 RTT	Y	Y	Y	
Address migration	Y	Y	Y	
Unreliable Delivery	N	N	Y	
Co-exist with other protocol	N	N	Y	
Streams				
Number of Streams (Note 1.)	1	1..n (Note 2.)	1..n	
Number of Control Streams	1	1	1	
Number of Data Streams	0	0..n (Note 2.)	0..n	
Broker initiated Stream	N	N	Y	
Stream flow control	N	Y	Y	
Stream prioritizion	N	Y (Note 3.)	Y	
Unidirectional stream	N	N	Y	TBD
Persistent sessions	Y	P (Note 4.)	Y	
Mitigate HOLB	N	Y	Y	
Send/Recv abortion	N	Y	Y	
Trackable Flows	N	N	Y	

Notes:

1. Number of concurrent streams

2. ‘n‘ defined by broker, suggested maximum 64k

3. Client set prioritizion.

4. On control stream only

8 Connections

8.1 Establishing a connection

QUIC connections are established as described in [RFC9000].

0-RTT support is optional.

Client SHOULD NOT create more than one QUIC connection to a given IP and UDP port.

8.2 Connection Keepalive

Connection keepalive SHOULD be performed on the QUIC transport. Both server and client maintain keepalive traffic on their own.

However, MQTT keepalive could still be used over QUIC, but note that if QUIC connection keepalive is set,
the connection idle timeout SHOULD be greater than the MQTT keepalive interval to prevent connection idle
shutdown while sending the MQTT.PINGREQ.

8.3 Connection termination

8.3.1 Graceful shutdown

Graceful shutdown only requires graceful shutdown of the control flow, other types of flows could be shut down gracefully or aborted. See flow shutdown section.

Connection graceful shutdown could be used for

Broker:

1. redirect the client to the new server

2. prevent MQTT WILL message from being sent.

Client:

1. clear session states

2. set a new session expiration time.

There is no graceful shutdown defined by the QUIC protocol.

In MQTT-next, if either endpoint wishes to gracefully disconnect,
it MUST send MQTT.DISCONNECT over the control stream with a reason code explicitly set in the Disconnect Reason Code.
Then it MUST terminate the control flow gracefully.

Any MQTT packets received before the control stream is closed SHOULD be properly handled.

After closing the control stream, an endpoint MUST shutdown the connection. Either explicitly (informing the peer) or silently (without informing the peer).

MQTT defines graceful shutdown with the stream shutdown reason code: NO_ERROR.

If MQTT coexists with http/3, the http/3 graceful shutdown procedure must also be followed.

1. Graceful shutdown initiated by the client:

   Client MUST first send MQTT.DISCONNECT over control flow
   AND then MUST wait for control flow graceful shutdown to complete
   AND then Client MAY shutdown the connection by starting the connection Immediate shutdown of the QUIC protocol
   OR the client MAY terminate the connection locally without notifying the peer.

   Client MUST discard all MQTT packets received from the Broker after sending the MQTT.DISCONNECT.

   If the client receives a QUIC CONNECTION_SHUTDOWN FRAME before completing the control flow graceful shutdown procedure
   then the graceful shutdown procedure will fail.

   Client MAY timeout waiting for a control flow graceful shutdown to complete, it MAY start an immediate connection shutdown procedure with code ERROR_DISCONNECT_TIMEOUT, then the Connection graceful shutdown is failed.

If the server receives MQTT.DISCONNECT via control flow,
it MAY attempt to gracefully shut down other flows by processing all received MQTT packets
AND if MQTT coexists with other protocols, it MUST wait for the other protocol to gracefully shutdown.
AND server MUST initiate control flow graceful shutdown.
AND server SHALL not send MQTT messages on any flows.
AND server MAY initiate the QUIC protocol's immediate disconnect procedure OR silently disconnect locally without notifying the peer.

2. Graceful shutdown triggered by the server:

   Server MUST first send MQTT.DISCONNECT via control flow
   AND then MUST wait for the control flow graceful shutdown to complete
   AND server MAY initiate the QUIC protocol's immediate connection termination procedure OR silently terminate the connection locally without notifying the peer.

8.3.2 Abnormal connection shutdown

Abnormal connecion shutdown is the shutdown of a connection that is not graceful.

Abnormal connecion shutdown does not require peers to cooperate.

The following conditions can trigger abnormal connection shutdown.

- Aborted control flow shutdown
- Immediate connection shutdown triggered locally by the application.
- Immediate shutdown triggered remotely without completing the control flow Graceful Shutdown
- Idle connection.
- Other unrecoverable transport errors such as device failure, OS failure, unhandled network changes.

8.3.3 Sending unreliable datagrams over the connection

The QUIC extension RFC 9221 introduces unreliable datagrams, allowing applications to transmit data over a QUIC connection
with an emphasis on speed over guaranteed delivery.

This offers benefits for real-time data and scenarios where occasional losses are acceptable.

Negotiation:

Support for unreliable datagrams is negotiated during the initial QUIC handshake transport parameter defined in RFC9221,
This allows both endpoints to agree on using datagrams before transmission.

MQTT packets can be directly encoded within the datagram payload for efficient transfer.

MQTT packet can be encoded in the payload of unreliable datagram.

Messages with QoS values greater than or equal to 0 MAY be sent as unreliable datagrams.

While offering flexibility using this mechanism implies relaxing the QoS guarantees associated with that message.

- Sender Responsibilities:

  The unreliable datagram is ACK-eliciting, the sender application MAY know if the datagram is received, lost or possibly lost,

and the application MAY choose implement appropriate loss detection and recovery mechanisms.

- Receiver Expectations:

Be prepared to receive datagrams out of order and potentially duplicated.
Implement mechanisms to handle these eventualities, such as deduplication based on unique identifiers or application-specific context.

Also the unreliable datagram may not be sent when the connection is alive, common Failure Scenarios:

- Unsupported Feature:

  If the peer does not support unreliable datagrams, sending attempts will fail with an appropriate error indication.

Applications should handle this scenario gracefully and switch to alternative communication channels, such as stream-based flows.

- Flow Control Limitations:
  Unreliable datagrams, like other QUIC data, are subject to flow control restrictions.
  If available flow control limits are exceeded, sending attempts will fail. In such cases, applications should either wait for more
  flow control credits or consider alternative channels that have sufficient capacity to accommodate the datagram transmission.

- MTU Size Constraints:
  If the datagram size exceeds the Maximum Transmission Unit (MTU) of the path, sending will fail.
  Applications can address this by either fragmenting application payload into smaller segments that comply with the MTU or exploring alternative
  channels that can handle larger payloads without fragmentation overhead.

MQTT-next defines three types of datagram payloads

1. Non-MQTT control packet datagram

   First byte must be 0x00 to distinguish from MQTT packet

2. MQTT control packets

3. Zero length datagram

   The use of zero length datagram should be allowed.

   The application could handle or ignore the UD with payload of 0 length.

   The function of the zero length datagram is implementation specific.

8.3.4 Connection downgrade

If the QUIC handshake fails or timed out, the client SHOULD downgrade the protocol to reconnect to the TCP/TLS endpoint.

The client SHOULD NOT downgrade from QUIC to plain-text TCP.

8.3.5 Discovering and upgrading

The client could learn that the server supports MQTT-next via ALPN during the TCP/TLS handshake, so the upgrade is possible
via QUIC connection to the same endpoint and port before the client sends the MQTT.connect

control message over TCP/TLS.

NOTE, When the client transmits the MQTT.connect packet to the server using both TCP-based transport and QUIC transport,
precedence is given to the latter connection established, the latter connection will take over the session.

9 MQTT Flows

The term `flow` is used in MQTT-next to distinguish the term `stream` in the QUIC protocol.

@NOTE
The stream id in QUIC protocol isn't transparent to the application, as stated in RFC9000:

> A stream ID that is used out of order results in all streams of
> that type with lower-numbered stream IDs also being opened.

Flows are the abstraction of concurrent logical streams in multistream advanced mode.

MQTT Flow provides reliable, ordered unidi/bidi transport for MQTT packets.

There may be one or more flows in a connection between two endpoints.

The flow header identifies the type of flow.

Application operates flows:

- Start new flow
- Start a flow with same flow id that was gracefully shutdown previously.
- Recover aborted flows with either a clean state or preserved state.
- Shutdown flows gracefully

  Terminate flows in an orderly manner.
- Abort the flow

  Immediately discontinue communication on a flow which could be
  abort sending, abort receiving or abort both sending and receiving.

- Refresh the flow

  Replace the stream of the flow with a new stream.

- Limit the number of flows.

- Flow control each flow in bytes.

  The maximum number of flows is limited by the connection flow control per implementation.

9.1 Flow usages

9.1.1 Control Flow

A single Control Flow must be initiated by the client per connection. All session-related information must be exchanged through this flow.

See 9.19

9.1.2 Data Flow

Both client and server can exchange pub/sub application data over the data flow.

If an MQTT.PUBLISH message needs to be sent for matching subscriptions, it must be sent over the data flow where it is subscribed.

The subscriber must expect messages for subscriptions from the same flow it subscribed to. Flows are independent, with no shared states between them. Therefore, multiple subscriptions to the same topic in different flows but in the same connection are possible, even with the same subscription ID.

If an MQTT.PUBLISH message needs to be sent as a publisher, it may be sent over the control flow, an existing data flow, or a new data flow.

As QoS > 1 messages track delivery states in the Flow State, the MQTT.PUBACK, MQTT.PUBREL, and MQTT.PUBCOMP messages for the same MQTT.PUBLISH message must be exchanged in the same data flow.

The client may send an MQTT.PINGREQ message to verify the availability of the application service on the server side. The server must respond with an MQTT.PINGRESP message through the same data flow where it received the PINGREQ. (TBD: This API could be negotiated in the flow header)

The server MAY send predefined subscription data to the client through a dedicated server-initiated data flow.

If the client's flow control limit does not allow for a dedicated server-initiated flow, the server SHOULD attempt to negotiate with the client for increased flow allowance by sending QUIC.DATA_BLOCKED

The server MUST not send predefined subscription data through any other client flows until the client grants the requested flow increase. This ensures respect for the client's resource limitations and prevents potential interference with existing client traffic.

See 9.19

9.2 Flow and stream mapping

A flow can use one QUIC bidi stream.

A flow can use one QUIC unidi stream or [TBD] a pair of QUIC unidi streams.

9.3 Flow ownership

The flow is owned by the endpoint which starts it.

The owner takes responsibility for the stream lifecycle, including startup, shutdown, restart after reconnect,
error recovery. This avoids race conditions or leaving unused streams.

9.4 Flow ID

Each flow has a `FlowID`, the FlowID is picked by initiator.

The FlowID is unique within the MQTT session.

FlowID is a Variable-Length Integer.

The least significant bit of the FlowID identifies if it is a server flow to avoid FlowID collision between client and server,
and the owner of the flow MUST ensure the the bit is correctly set.

9.5 Flow Type

In order for MQTT to coexist with other protocols on the same QUIC connection,
MQTT-next uses defined (see IANA) flow types to distinguish from the other protocols.

9.6 Flow Header

The flow header is the first few bytes used by both endpoints to identify the flow and gather information for using the flow.

@NOTE, the 'Variable-Length Integer Encoding' (i) in the flow header is defined in RFC 9000 and not the "Variable Byte Integer" in the MQTT specification.

@TODO, maybe simplify it by reusing 'MQTT.CONNECT'

Stream Header Formats:

9.6.1 Control Flow Stream header

```
control_flow_header {
  Flow_type(i) = 0x11,
  Flow_id(i): 0x00,
  Flow_persistent_flag(8),
}
```

9.6.2 Client Data Flow Stream header

```
client_data_flow_header {
  Flow_type(i) = 0x12,
  Flow_id(i),
  Flow_expire_interval(i),
  Flow_flags(8),
  [Flow_optional_headers]
}
```

9.6.3 Server Data Flow Stream header

```
server_data_flow_header {
  Flow_type(i) = 0x13,
  Flow_id(i),
  Flow_expire_interval(i),
  Flow_flags(8),
  [Flow_optional_headers]
}
```

9.6.4 User defined Flow Stream header

```
user_data_flow_header {
  Flow_type(i) = 0x14,
  Flow_id(i),
}
```

9.7 Flow Expire Interval

Similar to the session expiry interval in MQTT.CONNECT packet, specifies the number of seconds both the client and server will retain the
flow state information after the flow terminates unexpectedly (abortive shutdown).

9.8 Flow Flags

```
flow_flags {
  clean(1),
  abort_if_no_state(1),
  err_tolerance(2),
  persistent_qos(1),
  persistent_topic_alias(1),
  persistent_subscriptions(1),
  optional_headers(1),
}
```

clean:
if it is a clean start of the flow, both endpoint MUST discard the previous persistent flow states.

abort_if_no_state:
If set and flow state is gone for any reason, peer MUST abort this flow with RC: ERROR_NO_FLOW_STATE
It is protocol error level 1 if both this flag and clean flag are set.
Local node could restart the flow with clean set to true afterwards.

persistent_qos:
if set, both endpoints must persistent QoS states.

persistent_topic_alias:
if set, both endpoints must persistent topic alias
if unset, both endpoints must not persistent topic alias that topic alias mapping does not survives from a flow shutdown.

persistent_subscriptions(1):
if set, both endpoints must persistent subscriptions and subscription ID.
It is protocol error level 1 if this flag is set in server flow.

optional_headers(1):
if set, optional_headers are set

9.9 Optional Headers

```
optional_headers {
   optional_header ...
}
```

```
optional_header {
   header_len(8),
   header(header_len),
}
```

Predefined Optional header here:

@TODO

9.10 Flow start

Both client and server can initiate new flows.

The acceptor which is the peer of the flow initiator must check if the flow header is valided and supported. If not, the stream
recv should be aborted with the error code defined in **Error Code**.

Flow Termination on Inactivity,
Both the client and server are able to unilaterally abort a flow using the ERROR_FLOW_OPEN_IDLE code if the flow remains idle after it has been started.
This includes the situation where a timeout occurs upon receiving a complete stream header without subsequent data within the designated timeframe.

Mismatch of initiator and flow type in control flow is protocol error level 0.

Mismatch of initiator and flow type in data flow is protocol error level 1.

9.11 Send/Recv over the flow

A bidi flow has two endpoints and each endpoint has one sender and one receiver.

The bytes are received as the same order as when they are sent by sender in the same flow and this is ensured by QUIC protocol.

Sender may fail to send over the flow when peer aborts the receiving.

Receiver may fail to receive from the flow when peer aborts the sending.

When the abortion happens, the application MUST assume the data may or may not being handled properly at peer.

@NOTE, Some QUIC stacks may deliver bytes out of order to the application. However, these bytes will come with offsets that the application can use to recover the correct order.
to recover the order.

9.12 Flow expiration

MQTT Flow offers resilience to both QUIC connection interruptions and QUIC stream abortion.

Flows can survive disconnections as long as session and the flow are not expired.

When the flow is expired, the flow state MUST be removed from session state.

When the session is expired, all flow states associated with it will be expired.

The ‘flow_expire_interval‘ in the stream header defines for how long should the flow expire after abortive shutdown.

9.13 Flow Termination (Shutdown)

The flow termination could be triggered by either endpoint gracefully (clean) or aborting.

If graceful shutdown is triggered, it MAY end with abortive shutdown.

If abort is triggered, it MUST terminate with abortive shutdown.

Flow state MUST be removed from session state if gracefully terminated.

Flow state MUST NOT be removed from session state if it is aborted if the flow hasn't expired. @TODO what if app crash?

In the case of aborted termination, the sender MUST assume that the messages it has sent will be unhandled or handled, and for the receiver it is up to the implementation to decide how to deal with the received but unhandled data.

9.13.1 Flow graceful termination.

The importance of graceful shutdown is to ensure the sent data are received and processed by peer to reduce the chance of getting into undetermined state, reduce
the retransmittions after flow recover and last avoid data transmission get cancelled due to connection close.

~~The flow owner~~ Either endpoint can trigger the graceful shutdown of the flow by sending a QUIC STREAM FRAME with FIN flag.

The flow owner MUST finish sending a complete MQTT packet before starting the graceful shutdown procedure.

~~It is protocol error level 0 if the graceful shutdown of the flow is not initiated by the flow owner~~

It is protocol error level 2 for data flow and protocol error level 0 for control flow if the sender terminates the flow with an incomplete MQTT packet.
The recipient MUST reset the flow with APEC: ERROR_IMCOMPLETE_PACKET. (When FIN is set the recv size is known).

The graceful flow shutdown is completed ONLY when the other endpoint also terminates the stream by sending a QUIC STREAM FRAME with FIN flag set.

The receiver SHOULD ensure all received messages are processed before terminating the stream.

9.13.2 Flow abortive termination.

If the flow isn't terminated gracefully, it is abortive termination.

Abortive termination is triggered when at least one of the following events occurs

1. The sender aborts the transmission by sending a QUIC RESET_STREAM_FRAME.

2. Receiver aborts receive by sending a QUIC STOP_SENDING frame.

3. The sender receives the QUIC STOP_SENDING FRAME from the receiver.

4. Receiver receives QUIC RESET_STREAM FRAME from sender.

5. The connection is closed before the stream is properly terminated.

9.14 Flow takeover

Flow takeover is when the old QUIC stream in use by the flow is replaced with new QUIC stream in the middle of data transmission.

Flow takeover can only be triggered by the flow owner and takeover is done with the same Flow ID.

The new stream MUST have high order stream id of the same type. Greater QUIC Stream ID of the same QUIC stream type always takes precedence.

Flow takeover is used in the following cases

- To discard the obsolete data being transferred

- To update the stream priority in local stack.

- To refersh flow props, such as flow expire interval.

- To recover from the application error

- @TODO could we define graceful takeover without data loose?

The flow takeover could be triggered unintentionally due to the nature of parallelism of QUIC streams where the message of restart the
flow with new stream arrives before the abortion of the old QUIC stream.

By nature of the QUIC protocol, the stream owner MUST assmue the data sent before the takeover MAY or MAY NOT be handled by peer.

The flow takeover has the side effect that the owner aborts both sending and receiving, and the acceptor of the stream
MUST unconditionally abort its send/recv on the old stream.

The incomplete MQTT packet in the buffer at both ends MUST be discarded that is the data cannot survive from the old stream to the new stream.

The stream owner MUST ensure that it has sufficient flow control credits before starting the takeover process.

9.15 Flow Recover

Flow Recover means that a previously aborted flow identified by Flow ID is restarted from their preserved state.

There are two cases where flow recovery happens:

1. In cases where a flow was deliberately aborted for any reason, the owner of the flow can initiate
   a recovery request to revive it with its previous state.

2. When a connection is interrupted and later re-established, flows that were active before the disconnection
   can be recovered if their state is preserved.

Flow recover success only when both endpoints hold the preserved state.

The owner of the flow is responsible for restarting the flow with the ‘clean‘ bit in Flow flag MUST set to False to recover the flow.

If the receiver cannot successfully recover the flow state for any reason AND the ‘abort_if_no_state‘ bit is set,
it MUST abort the flow with the ERROR_NO_FLOW_STATE error code.

If the receiver cannot successfully recover the flow state for any reason AND the ‘abort_if_no_state‘ bit is unset,
it MUST NOT abort the flow with the ERROR_NO_FLOW_STATE error code. It is considered a protocol error (level 0) if receiver
does not follow this and the connection should be abored with ERROR_PROTOCOL_L0.

Repeated recovery attempts:

It is considered a protocol error (level 0) to attempt recovering a flow again if a previous attempt failed
with the ERROR_NO_FLOW_STATE error code. In such cases, the connection SHOULD be aborted with the
ERROR_TOO_MANY_RECOVER_ATTEMPTS error code.

9.16 Discard the Flow state at peer

Alternatively, instead of recovering the flow after abort, the flow owner could send a QUIC_STREAM_FRAM
with the FIN flag set, clean_start set,
and persistent flag cleared in the stream header to discard the flow state at the remote endpoint.

The stream acceptor MUST discard the flow state and complete the stream graceful shutdown by sending a QUIC_STREAM_FRAME
with the FIN flag set and zero-length data.

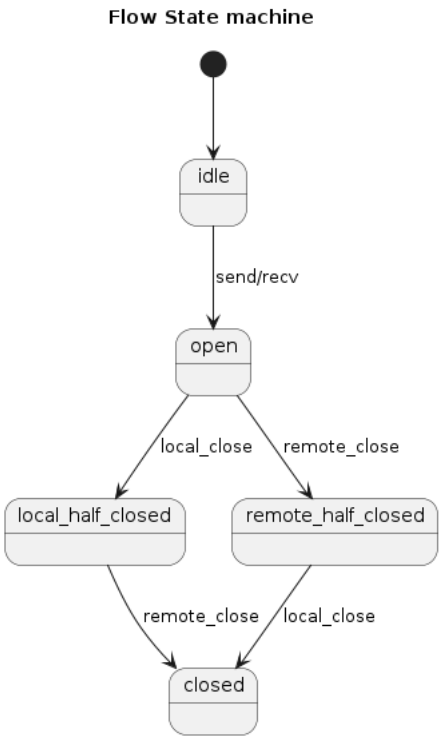
9.17 Flow state machine

Each endpoint has one sender and one receiver.

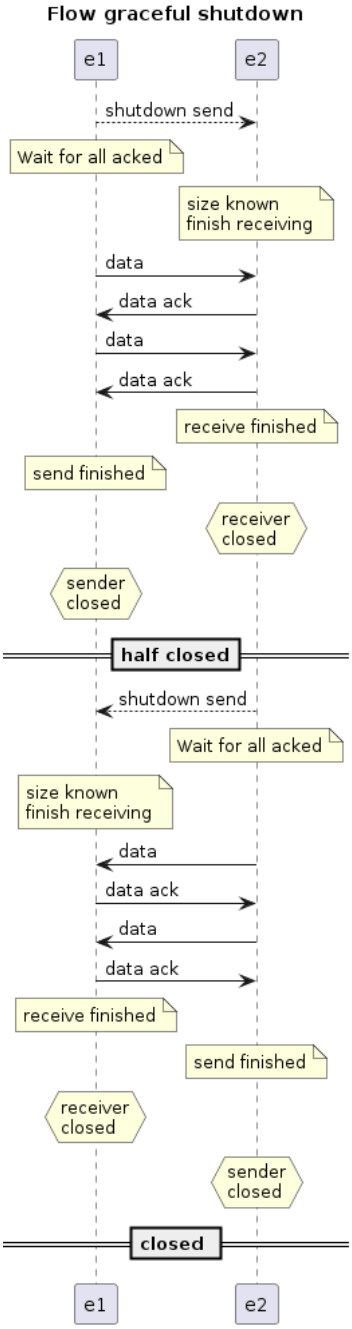
Sender state machine, refer to 3.1 in RFC9000
Receiver state machine, refer to 3.2 in RFC9000

Endpoint composed state:

Sending Part	Receiving Part	Composite State
No Stream / Ready	No Stream / Recv (*1)	idle
Ready / Send / Data Sent	Recv / Size Known	open
Ready / Send / Data Sent	Data Recvd / Data Read	half-closed (remote, graceful)
Ready / Send / Data Sent	Reset Recvd / Reset Read	half-closed (remote)
Data Recvd	Recv / Size Known	half-closed (local, graceful)
Reset Sent / Reset Recvd	Recv / Size Known	half-closed (local)
Reset Sent / Reset Recvd	Data Recvd / Data Read	closed (aborted)
Reset Sent / Reset Recvd	Reset Recvd / Reset Read	closed (aborted)
Data Recvd	Data Recvd / Data Read	closed (graceful)
Data Recvd	Reset Recvd / Reset Read	closed (aborted)



9.18 Seq chart of graceful shutdown



9.19 MQTT Packet and Flow mappings

Message direction follows MQTT 5.0

MQTT Packet	Control flow	Client Data flow	Server Data Flow	Unreliable Datagram
CONNECT	YES	NO	NO	NO
CONNACK	YES	NO	NO	NO
PUBLISH	YES	YES	YES	YES
PUBACK	YES	YES	YES	YES
PUBREC	YES	YES	YES	YES
PUBCOMP	YES	YES	YES	YES
PUBREL	YES	YES	YES	YES
SUBSCRIBE	YES	YES	NO	YES
SUBACK	YES	YES	NO	YES
UNSUBSCRIBE	YES	YES	NO	YES
UNSUBACK	YES	YES	NO	YES
PINGREQ	YES	YES	YES	NO
PINGRESP	YES	YES	YES	NO
DISCONNECT	YES	NO	NO	NO
AUTH	YES	NO	NO	NO

9.20 Table of Flow Types

MQTT Types (id.)	dir	initiate by	Transport data
Control flow (0x11)	bidi	Client	MQTT control packet
Client flow (0x12)	bidi/unidi	Client	MQTT data packet
Server flow (0x13)	bidi/unidi	Server	Server assigned subscriptions
User-Defined flow (0x14)	bidi/unidi	Client/Server	Other protocol data

Note, type '0x1f * N + 0x21' are reserved
Note, control packet and data packet are redefined here

Flow could only be recoverd by the same initiator.

9.21 Flow State

@TODO, here does not even mention the flow props, same as in MQTT 5.

The flow state is associated with the FlowId, the flow state persists from connection and stream close.

The flow state is used to persist the send state of the flow, which includes

- Flow type (ownership and usage)
- Subscription
- topic alias

- Delivery state of QoS > 0 messages sent.

Each endpoint of a flow maintains its own flow state as a minimum persistence:

9.21.1 Client side

- Delivery state of QoS >0 messages sent.
- Topic alias

9.21.2 Server side

- Subscriptions and subscription ID
- Topic alias
- Delivery state of QoS > 0 messages.
- Buffered QoS >0 messages, QoS 0 optional.
- Flow Expiry Time

10 Session

@TODO, here does not even mention the session props, same as in MQTT 5.

10.1 Session State

The existence of the session.

Session State is associated with MQTT Client ID.

Session State contains the zero or many flow states.

Session state contains session expire interval.

Session State must be discarded when the connection is closed AND the session expire interval has passed.

If the session state is discarded, the flow states in the session are also discarded.

11 Error Handlings

MQTT-next is designed to be robust to application errors so that the connection could be maintained and the other application muxing the flow in the same connection are not affected by errors that are isolated.

Errors do not necessarily mean logical errors or protocol violations. It could also mean the cancellation of operations such as
aborting the transmission of a large payload, or cancelling a subscription that is no longer of interest as a shortcut to sending an unsubscribe.

There are three levels of protocol error:

- Protocol Error Levels

- Protocol error level 0

This is a serious error that cannot be violated or the connection cannot be served by the broker.

The connection MUST be closed.

For other errors, it is up to the implementation whether to close the connection by notifying the peer or to close silently.

1. Protocol error level 1

The error is isolated in the specific flow, but the flow state MUST be discarded because it is impossible to maintain the state
or the error could lead to inconsistent states.

1. Protocol error level 2

Not a serious error, most likely could be recovered with a retry or the error is isolated in the specific flow.

The handling of protocol error level 2 could be negotiated between the two endpoints or decided by implementation.

The flow state is maintained but the flow is aborted and a restart is required for recovery.

The endpoints aborting the flow MUST abort the flow with a reason sent to the peer.

The endpoint MAY gracefully shut down or abort another flow as a side effect of a protocol error level 2.

12 Error Code

12.1 Application Error Code on Connection

Error code used in the QUIC CONNECTION_CLOSE Frame

Error Name	Code	Reuse MQTT 5	Meaning
NO_ERROR	0x00		
ERROR_TLS_ERROR	0xB1		TLS handshake success but
ERROR_UNSPECIFIED	0xB2		Default UNSPECIFIED erro
ERROR_TOO_MANY_RECOVER_ATTEMPTS	0xB3		Too many attemps to recove
ERROR_PROTOCOL_L0	0xB4		Protocol Error Zero 0

12.2 Application Error Code on Stream Flow

Error code used in QUIC RESET_STREAM FRAME

Error Name	Code	Packet	Discard State	Meaning
NO_ERROR	0x00		N	NO ERROR
ERROR_NO_FLOW_STATE	0xB3		N/A	FLOW STATE does not
NOT_FLOW_OWNER	0xB4		N	Only FLOW owner is all
ERROR_STREAM_TYPE	0xB5		N	Unsupported stream typ
ERROR_BAD_FLOW_ID	0xB6		Y	FlowID and FlowType n
ERROR_PERSISTENT_TOPIC	0xB7		N	Persistent topic alias uns
ERROR_PERSISTENT_SUB	0xB8		Y	Persistent subscription u
ERROR_OPTIONAL_HEADER	0xB9		Y	Optional Headers unsup
ERROR_IMCOMPLETE_PACKET	0xBA		N	Receiver abort graceful s
ERROR_FLOW_OPEN_IDLE	0xBB		N	FLOW is idle, no data a
ERROR_FLOW_CANCELLED	0xBC		Y	FLOW operation is canc
ERROR_FLOW_PACKET_CANCELLED	0xBD		N	FLOW operation is canc
ERROR_FLOW_REFUSED	0xBE		N	FLOW is refused
ERROR_DISCARD_STATE	0xBF		Y	The entire FLOW state
ERROR_SERVER_PUSH_NOT_WELCOME	0XC0		Y	Server Push flow is not w
ERROR_NO_FLOW_STATE	0xC1		Y	Could not recover the flo

12.3 Error Code in MQTT Packet

Refer to MQTT 5.0, 2.4 Reason Code.

This applies to the datagram as well.

13 Limitations

1. To resume a multistream session after fallback to TCP based transport needs extra work in this spec to reuse TCP connection for all the streams.

14 IANA Considerations

@TBD

15 Opportunities

15.1 Enable MQTT Stream publish mode.

New mode that MQTT publish a message with undermined payload len.

The len of message payload could exceed the max size of a messages (256MB) defined in MQTT 5.0 protocol.

A MQTT messages with payload len 0 could be used for stream mode which length of the payload is undefined such as
tail logging of a log file. This also needs to assign a specific stream type. The flow data would look like:

```
MQTT_stream {
  stream_header,
  mqtt_pub_fixed_header,
  mqtt_pub_var_headers,
  payload_stream ...
}
```

When the length of stream is determined such as producer of the stream get EOF(end of file),
the flow owner should use graceful shutdown to terminate the sending, with determined length of data.
And the receiver MUST ack the message for QoS >0 before gracefully shutdown the stream.

16 Open Questions

1. Another alt. of starting a flow is to send MQTT.connect right after the flow header.
   So we have very short flow header + normal MQTT.connect message which contains alls the flow specific

params such as flow expire interval, max packet size.